WEB DEVELOPMENT LABORATORY WEB EXERCISES

VERSION CONTROL WITH GIT AND GITLAB

1. Create your own project in GitLab:
    a. Start with a local repository and a simple file (README.md or hello.txt).
    b. Add the remote repository in GitLab, make commits and push your changes.
2. Create a GitLab project in a team (2 members):
    a. Work together with a partner in the same GitLab project.
    b. Each member creates a different text file (e.g., a.txt, b.txt) and shares it with the other using commit, push and pull on the command line.
    c. Also test conflict resolution by editing the same file in parallel.
3. Use the GitLab project created by the instructor (individual):
    a. Clone the repository assigned to you by the instructor.
    b. Add new files and organize them into logical folders.
    c. Practice commits with descriptive messages and push the changes.
    d. Create a new branch, make some changes, and then merge it back into main.
    e. Document in the README.md the commands used and the steps followed.

CONTAINERS AND ORCHESTRATION

4. Run an interactive Ubuntu container with shell access.
5. Run the Nginx Web server and access the main page via browser.
6. Create a simple container based on Alpine Linux:
    a. Copy a hello.txt file into the container.
    b. Configure the Dockerfile to display the contents of the file at startup (cat hello.txt).
7. Create and run a simple Nginx container:
    a. Build the image from the Dockerfile.
    b. Validate that the container serves the index.html file in the browser.
8. Create a simple Python application in a container:
    a. Use the Dockerfile and files (app.py).
    b. Build the image and confirm that the application prints "Hello World".
9. Configure docker-compose.yml to use a .env file:
    a. Define environment variables such as DB_USER, DB_PASSWORD, and DB_NAME.
    b. Confirm that the variables are correctly applied to MariaDB.
10. Serve a static page with Nginx:
    a. Configure a docker-compose.yml that starts the Nginx service.
    b. Mount the site/ folder with the index.html file.
    c. Validate that the page "Hello Docker" is accessible at http://localhost:8080.
11. Use a .env file in Docker Compose:
    a. Create a .env file with the variables POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_HOST.
    b. Implement a small Python application (CLI) that, on startup, reads these variables (for example, using os.getenv) and prints the values as well as the connection string in the format: postgresql://<user>:<password>@<host>/<db> (use the program provided in the slides)
    c. Create the application's Dockerfile and configure docker-compose.yml to use env_file: .env, set depends_on: [db], and, in a development environment, mount the code using volumes if necessary.
    d. Place all files in the provided Git repository, make the appropriate commits, and add a README.md with the executed commands and the output of the run.

12. Inspect and interact with running containers:
    a. Start an Nginx container in the background (docker run -d -p 8080:80 nginx:alpine).
    b. List running containers with docker ps and identify the CONTAINER ID.
    c. Enter the container with docker exec -it <container_id> sh and explore the /usr/share/nginx/html directory.
13. View and analyze logs of services in Docker Compose:
    a. Start the services defined in docker-compose.yml.
    b. Use docker compose logs -f to follow the logs in real time.
    c. Stop the services with docker compose down and verify that the logs are no longer updated.
14. Test communication between services on an internal network:
    a. Configure a docker-compose.yml with two services: an app (Python or Alpine) and a db (PostgreSQL).
    b. Verify that the app can communicate with the db through the service name (db) without using the IP.
    c. Test isolation by confirming that the db is not directly accessible via the browser on localhost.

INTRODUCTION TO PYTHON

15. Configure a virtual environment with venv and requirements.txt.
    a. Create `.venv/`, activate it and install versions: `Django~=5.0`, `djangorestframework~=3.0`.
    b. Add a `.gitignore`.
    c. Push the project to a remote repository.
    d. Create a script `check_env.py` with `main` that prints `django.get_version()`.
16. Practice basic I/O and types with input() and print().
    a. Read name and age; print a formatted message using f-strings.
    b. Show variable types with `type()`.
    c. Convert age to integer with validation (error message if it fails).
17. Conditions and loops, each task in a separate function.
    a. Read a number and indicate whether it is even.
    b. Maximum of three: read three numbers and print the largest (without using `max()` in the first version).
    c. Countdown from N to 0 with `while`.
18. Separate code into modules.
    a. Create `utils.py` with the functions from exercise 17.
    b. In `main.py`, import and use `from utils import ...`.
19. Count word occurrences in a sentence (frequency dictionary) and display the results.
20. Implement a command-line application to manage tasks.
    a. Use an interactive menu to add, complete tasks, and list them.
    b. Each task is represented by an id, a title, and a status (pending or completed).
21. Develop a command-line application to manage a shopping list.
    a. Use an interactive menu.
    b. Each item is represented by a product name, a quantity, and whether it has been purchased or not.
    c. A provided Git repository must be used, and at least 10 descriptive commits must be created.
    d. Include a README.md that documents the algorithm used and the real output of an example execution.

WEB FUNDAMENTALS

22. Exploring HTTP requests in the browser

    a. Open the website https://www.ipb.pt.

    b. Enable the Developer Tools (DevTools → Network tab).

    c. Reload the page (Ctrl + R or F5).

    d. Observe:

        i. The initial request (Request URL and Method).

        ii. The status codes (e.g., 200, 304).

        iii. The response headers (Content-Type, Cache-Control, ETag, etc.).

        iv. The loading time and the transferred size.

23. Exploring HTTP requests with Postman (or curl)

    a. Make a GET request to:
    https://www.ipb.pt/cacheapi/api/menu/protocolosprojetos

    b. Observe:

        i. The status code.

        ii. The body format (application/json).

        iii. The headers (Cache-Control, ETag, Date, etc.).

    c. Test the same request from the command line:
    curl -I https://www.ipb.pt/cacheapi/api/menu/protocolosprojetos

DJANGO ARCHITECTURE AND CONFIGURATION

24. Perform the following tasks, creating a local repository and committing as needed. When instructed, publish to the remote repository provided by the instructor.

    a. Create the working folder order_system/ and initialize Git.

    b. Create a Python virtual environment (e.g., python -m venv .venv) and activate it.

    c. Create the file docker/requirements.txt with the following dependencies:

```
Django~=5.0
psycopg[binary]~=3.0
debugpy~=1.0
```

    d. Install the dependencies in the local virtual environment:

```
pip install -r docker/requirements.txt
```

    e. Create an empty README.md file in the root of order_system/.

    f. Create .gitignore and add at least: .venv/, .env, .env.db, __pycache__/, staticfiles/ (or use a suitable template).

    g. Create .env (Django variables):

```
DEBUG=1
SECRET_KEY=<to-be-generated>
ALLOWED_HOSTS=*
```

    h. Create .env.db (Postgres variables — used by both services):

```
POSTGRES_ENGINE=django.db.backends.postgresql
POSTGRES_DB=database
POSTGRES_USER=user
POSTGRES_PASSWORD='put a password'
POSTGRES_HOST=db
POSTGRES_PORT=5432
```

    i. Generate the SECRET_KEY and paste it into .env::

```
python -c "from django.core.management.utils import get_random_secret_key as g; print(g())"
```

j. Create .env.example and .env.db.example with placeholders (no secrets).

k. Create the Django project inside the folder oms/, running from the order_system/ root:

```
django-admin startproject oms
```

l. Start the application server, open it in the browser, and add a screenshot to README.md:

```
python oms/manage.py runserver
```

25. Continue from the previous exercise and perform the following tasks, committing as needed. When instructed, publish to the remote repository indicated by the instructor (replacing the previous one if necessary).

a. Create docker/Dockerfile:

```
FROM python:3-slim

WORKDIR /home/app/web
ENV PYTHONDONTWRITEBYTECODE=1 \
   PYTHONUNBUFFERED=1 \
   PIP_DISABLE_PIP_VERSION_CHECK=1

RUN apt-get update -y && apt-get install -y --no-install-recommends \
   build-essential  && \
   rm -rf /var/lib/apt/lists/*

RUN pip install --upgrade pip

COPY ./requirements.txt /requirements.txt
RUN pip install --no-cache-dir -r /requirements.txt

COPY ./entrypoint.sh /entrypoint.sh
RUN chmod 0755 /entrypoint.sh

CMD [ "/entrypoint.sh" ]
```

b. Create docker/entrypoint.sh:

```
#!/bin/bash

python manage.py migrate --noinput
python manage.py collectstatic --noinput
python manage.py runserver 0.0.0.0:8000
```

c. Create docker-compose.yml:

```
services:
  oms_web:
    container_name: oms_web
```

```
    build:
      context: ./docker
      dockerfile: Dockerfile
    restart: unless-stopped
    env_file:
      - ./.env
      - ./.env.db
    ports:
      - 5678:5678
      - 8000:8000
    volumes:
      - ./oms/:/home/app/web/
  oms_db:
    container_name: oms_db
    image: postgres:16
    restart: unless-stopped
    env_file:
      - ./.env.db
    volumes:
      - oms_data:/var/lib/postgresql/data/
    ports:
      - 5432:5432

volumes:
  oms_data:
```

d. Edit order_system/oms/oms/settings.py to use variables from .env and .env.db (only these excerpts):

```
import os

DEBUG = int(os.getenv("DEBUG", 0))
SECRET_KEY = str(os.getenv("SECRET_KEY", "unsafe-dev-secret"))
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "*").split(",")

DATABASES = {
    "default": {
        "ENGINE": os.getenv("POSTGRES_ENGINE", "django.db.backends.postgresql"),
        "NAME": os.getenv("POSTGRES_DB", "database"),
        "USER": os.getenv("POSTGRES_USER", "user"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD", "password"),
```

```
        "HOST": os.getenv("POSTGRES_HOST", "db"),
        "PORT": os.getenv("POSTGRES_PORT", "5432"),
    }
}

STATIC_URL = "static/"
STATIC_ROOT = os.path.join(BASE_DIR, "staticfiles")
```

e. Build images, start services, and inspect logs:

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f oms_web
```

f. In README.md, paste excerpts of web and database logs, and include a screenshot of accessing http://localhost:8000.

26. Continue from the previous exercise to access the Django admin page. Perform the following tasks, committing as needed. When instructed, publish to the remote repository indicated by the instructor (replacing the previous one if necessary).

    a. Create a superuser:

```
docker compose exec oms_web python manage.py createsuperuser
```

    b. Access http://localhost:8000/admin. Add a screenshot of /admin to README.md.

    c. List database tables and paste the result of \dt into README.md. First execute:

```
docker compose exec oms_db bash
```

    And then:

```
psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "\dt"
```

    d. Check the contents of the staticfiles/ folder. Add a partial listing to README.md.

27. Continue from the previous exercise to create a test page and configure debugging in VS Code. Commit as needed. When instructed, publish to the remote repository indicated by the instructor (replacing the previous one if necessary).

    a. Create the application shop (inside the web container):

```
docker compose exec oms_web python manage.py startapp shop
```

    b. Enable the app in Django by editing order_system/oms/oms/settings.py and adding "shop" in INSTALLED_APPS:

```
INSTALLED_APPS = [
    # ...
    "shop",
]
```

    c. Create a minimal Hello World view in order_system/oms/shop/views.py:

```
# file: shop/views.py
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("Hello, world!")
```

    d. Map the /test/ route in the project URL dispatcher by editing order_system/oms/oms/urls.py:

```
path("test/", hello_world, name="hello_world"),
```

    e. Configure VS Code for remote debugging by editing order_system/.vscode/launch.json:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Remote Attach",
```

```
      "type": "debugpy",
      "request": "attach",
      "connect": { "host": "localhost", "port": 5678 },
      "pathMappings": [
        {
          "localRoot": "${workspaceFolder}/oms",
          "remoteRoot": "/home/app/web"
        }
      ]
    }
  ]
}
```

f. Activate the debugpy listener when DEBUG=1, by editing order_system/oms/manage.py and, before execute_from_command_line(sys.argv), insert:

```
from django.conf import settings
if settings.DEBUG:
    if os.environ.get("RUN_MAIN") or os.environ.get("WERKZEUG_RUN_MAIN"):
        import debugpy
        debugpy.listen(("0.0.0.0", 5678))
        print("Attached remote debugger")
```

g. Restart the web service (if necessary):

```
docker compose restart oms_web
```

h. Place a breakpoint on the first line of hello_world().

i. In VS Code, start the Attach configuration (F5) and access:

```
http://localhost:8000/test/
```

j. Add to README.md a screenshot of VS Code showing execution stopped at the breakpoint during debugging.

VIEWS, TEMPLATES AND MODELS

28. Perform the following tasks, creating a local repository and making commits whenever you find it necessary. When indicated, publish it to the remote repository provided by the instructor.
    a. Copy the folder from the previous module's exercise into a new folder named order_system_6/.

    b. Inside the order_system_6/ folder, delete the .git/ folder to remove the previous module's git history.

    c. Initialize a new Git repository in the order_system_6/ folder:

    d. Configure the remote repository provided by the instructor.

    e. Create a new application, by running on the host (where the docker-compose.yml file is located):

```
docker compose exec oms_web python manage.py startapp products shop/products
```

    f. Open oms/oms/settings.py and register the new app. Since it is inside shop, the module name is shop.products:

```
INSTALLED_APPS = [
    # ...
    "shop",
    "shop.products",
]
```

    g. Create a class-based view to explore the request by editing shop/products/views.py and inserting:

```
from django.views import View
from django.http import HttpResponse

class RequestExplorerView(View):
    def get(self, request):
        # query params: /shop/products/request-info/?page=2&order=desc
        query_params = dict(request.GET)
        method = request.method
        path = request.path
```

```
    user = request.user.username if request.user.is_authenticated else "anonymous"
    client_ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")

    lines = [
        f"Method: {method}",
        f"Path: {path}",
        f"User: {user}",
        f"Query params: {query_params}",
        f"Client IP: {client_ip}",
        f"User-Agent: {user_agent}",
    ]
    return HttpResponse("\n".join(lines), content_type="text/plain")
```

h. Create (or edit) shop/products/urls.py:

```
from django.urls import path
from .views import RequestExplorerView

app_name = "products"

urlpatterns = [
    path("request-info/", RequestExplorerView.as_view(), name="request_info"),
]
```

i. Link the shop.products routes in the shop app by editing shop/urls.py to include the product routes:

```
from django.urls import path, include

urlpatterns = [
    path(
        "products/",
        include(("shop.products.urls", "products"), namespace="products"),
    ),
```

```
]
```

j. In oms/urls.py, ensure it contains:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("shop/", include("shop.urls")),
]
```

k. On the host:

```
docker compose restart oms_web
```

Then access:

```
http://localhost:8000/shop/products/request-info/
```

And test with query parameters:

```
http://localhost:8000/shop/products/request-info/?page=2&order=desc
```

You should see the request content displayed on the page.

l. Modify the RequestExplorerView to display the specific value of the page parameter when present in the request. Add the following line right after capturing query_params:

```
class RequestExplorerView(View):
    def get(self, request):
        # query params: /shop/products/request-info/?page=2&order=desc
        query_params = dict(request.GET)
        page = request.GET.get("page")  # new line
        method = request.method
```

```
    path = request.path
    user = request.user.username if request.user.is_authenticated else
"anonymous"
    client_ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")

    lines = [
        f"Method: {method}",
        f"Path: {path}",
        f"User: {user}",
        f"Query params: {query_params}",
        f"Page param: {page}",  # new line
        f"Client IP: {client_ip}",
        f"User-Agent: {user_agent}",
    ]
    return HttpResponse("\n".join(lines), content_type="text/plain")
```

Then, access the test route again:

```
http://localhost:8000/shop/products/request-info/?page=2&order=desc
```

The result should now include an additional line showing the value of the page parameter.